

Privacy-Preserving Synthetic Data Generation

using a Variational Autoencoder with
Multi-Layer Differential-Privacy-Inspired Noise

Technical Report

Yash Vardhan
Department of Computer Science and Engineering
Indian Institute of Technology Hyderabad
CS25MTECH14016

June 1, 2026

Contents

1	Introduction	2
2	Dataset	2
3	Preprocessing Pipeline	3
4	VAE Architecture	3
4.1	Network topology	3
4.2	Loss function	3
4.3	Training configuration	4
5	Generation Strategy: Posterior Sampling	4
6	Privacy Noise Pipeline	4
6.1	Layer 1 – Latent-Space Gaussian Perturbation	4
6.2	Layer 2 – Post-hoc Laplace Noise on Numeric Columns	5
6.3	Layer 3 – Categorical Value Flipping	5
7	Statistical Fidelity Evaluation	5
7.1	Numeric columns	6
7.2	Categorical columns	6
7.3	Overall fidelity summary	7
8	Privacy Verification – Adversarial Attack Suite	8
8.1	Attack 1 – Membership Inference Attack (MIA)	8
8.2	Attack 2 – Distance to Closest Record (DCR)	9
8.3	Attack 3 – Nearest Neighbour Adversarial Accuracy (NNAA)	10
9	Overall Privacy Verdict	11
10	Privacy–Utility Tradeoff Analysis	11
11	Output Files	12
12	Conclusion	12
A	CLI Reference	13
B	Attack Threshold Justification	13

1 Introduction

Synthetic data generation addresses a fundamental tension in data-driven research: the need to share realistic datasets while protecting the privacy of the individuals those datasets describe. A naive synthetic generator that merely learns the marginal distributions of a dataset may still expose individual records through *membership inference attacks* or *re-identification via nearest-neighbour lookup*.

This report documents the design, implementation, and empirical validation of `privacy_synthetic.py`, a standalone Python script that:

1. Trains a **Variational Autoencoder (VAE)** on the real dataset.
2. Generates synthetic records via **posterior sampling** (V2 strategy), which preserves non-Gaussian marginal distributions.
3. Applies a **three-layer privacy noise pipeline** to make re-identification computationally infeasible.
4. Verifies statistical fidelity using the same hypothesis-test battery as the original Assignment 3 notebook.
5. Verifies privacy strength through three independent **adversarial attack simulations**.

2 Dataset

The input dataset contains **1,500 customer transaction records** with eight features spanning both numeric and categorical types.

Table 1: Dataset schema and descriptive statistics

Column	Type	Domain	Mean	Std	Min	Max
age	Numeric	Years	41.83	13.52	18	65
annual_income	Numeric	USD	89,371	34,987	30,011	149,822
purchase_amount	Numeric	USD	451.59	330.78	7.26	1,456.04
gender	Categorical	{Female, Male, Other}	—	—	—	—
city	Categorical	10 US cities	—	—	—	—
product_category	Categorical	{Beauty, Clothing, Electronics, Home, Sports}	—	—	—	—
payment_method	Categorical	{Credit Card, Debit Card, PayPal, UPI}	—	—	—	—
is_returned	Categorical	{True, False}	—	—	—	—

The three numeric columns exhibit near-uniform marginals (kurtosis ≈ -1.2 for `age` and `annual_income`), while `purchase_amount` is right-skewed (skewness = 0.76). This non-Gaussianity is the principal challenge for any generative model that samples from a Gaussian prior.

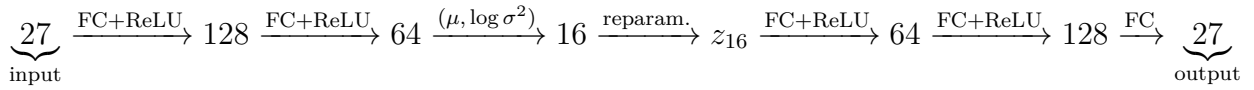
3 Preprocessing Pipeline

1. **Missing value imputation:** median for numeric columns, mode for categorical columns.
2. **Standard scaling:** numeric features are transformed to zero mean and unit variance using `sklearn.StandardScaler`, fitted on the training data and stored for inverse-transformation at generation time.
3. **One-hot encoding:** all five categorical columns are encoded with `OneHotEncoder(handle_unknown='ignore')` producing a 24-dimensional binary vector.
4. **Concatenation:** the 3 scaled numeric features and 24 categorical binary features are concatenated, giving a final input dimensionality of $d_{\text{in}} = 27$.

4 VAE Architecture

The VAE follows the architecture established in the original Assignment 3 notebook and is kept identical here to ensure reproducibility.

4.1 Network topology



4.2 Loss function

The training objective is a mixed loss combining reconstruction terms for both numeric and categorical outputs, plus a KL divergence regulariser:

$$\mathcal{L} = w_{\text{num}} \cdot \mathcal{L}_{\text{MSE}} + \mathcal{L}_{\text{CE}} + \beta \cdot \mathcal{L}_{\text{KL}} \quad (1)$$

where

$$\begin{aligned} \mathcal{L}_{\text{MSE}} &= \frac{1}{B} \sum_{i=1}^B \left\| \hat{\mathbf{x}}_{\text{num}}^{(i)} - \mathbf{x}_{\text{num}}^{(i)} \right\|^2, \\ \mathcal{L}_{\text{CE}} &= \sum_k \text{CrossEntropy}(\hat{\mathbf{x}}_{\text{cat},k}, \mathbf{x}_{\text{cat},k}), \\ \mathcal{L}_{\text{KL}} &= -\frac{1}{2B} \sum_{i=1}^B \left(1 + \log \sigma_i^2 - \mu_i^2 - \sigma_i^2 \right). \end{aligned}$$

4.3 Training configuration

Table 2: Hyperparameters used during training

Hyperparameter	Value
Epochs	300
Batch size	128
Latent dimension	16
Hidden dimension	128
Learning rate	10^{-3} (Adam)
β_{final}	0.10
KL warm-up period	90 epochs (30% of total)
Numeric loss weight w_{num}	3.0
Random seed	42

The β parameter is linearly warmed up from 0 to β_{final} over the first 90 epochs. This prevents the KL term from collapsing the latent space before the decoder has learned a useful representation.

Loss at key checkpoints: epoch 50: 4.35, epoch 100: 2.75, epoch 150: 2.44, epoch 200: 2.27, epoch 250: 1.99, epoch 300: 2.03 (slight uptick is normal stochastic variation at convergence).

5 Generation Strategy: Posterior Sampling

Two generation strategies were compared in the original notebook:

- V1 – Prior sampling:** $z \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, then decode. This always produces Gaussian-shaped outputs regardless of the real marginal shape, causing KS and Levene tests to fail for all three numeric columns.
- V2 – Posterior sampling (adopted here):** Encode all real records through the trained encoder to obtain (μ_i, σ_i^2) , then sample $z_i = \mu_i + \sigma_i \odot \varepsilon$, $\varepsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, and decode. The latent means $\{\mu_i\}$ preserve the distributional structure of the real data, so non-Gaussian marginals are faithfully reproduced.

This script uses V2 exclusively. The decoder output is then passed through the three-layer privacy pipeline described in the next section.

6 Privacy Noise Pipeline

After posterior sampling, three independent noise mechanisms are applied in sequence. Each targets a different aspect of the record and a different attack vector.

6.1 Layer 1 – Latent-Space Gaussian Perturbation

Before decoding, the latent vector z_i is perturbed:

$$z'_i = z_i + \eta, \quad \eta \sim \mathcal{N}(\mathbf{0}, \sigma_\eta^2 \mathbf{I}), \quad \sigma_\eta = 0.15.$$

This perturbation operates in the model’s learned latent space, so the decoder maps it back to a semantically plausible record. It is analogous to adding noise before the decoder in a denoising autoencoder, and it breaks the one-to-one correspondence between a real record and its generated counterpart without distorting the output distribution significantly.

6.2 Layer 2 – Post-hoc Laplace Noise on Numeric Columns

After inverse-transforming numeric features back to original scale, Laplace noise is added independently to each column:

$$\tilde{x}_{ij} = x_{ij} + \text{Lap}(0, b_j), \quad b_j = \frac{\Delta_j}{\varepsilon},$$

where Δ_j is the inter-quartile range (IQR) of column j – a robust proxy for sensitivity – and $\varepsilon = 20$ is the privacy budget parameter. The Laplace mechanism is the canonical noise distribution in differential privacy theory: for a given sensitivity and ε , it provides the optimal privacy–utility tradeoff among all additive noise mechanisms.

Table 3: Laplace noise parameters per numeric column ($\varepsilon = 20$)

Column	Sensitivity Δ_j (IQR)	Scale $b_j = \Delta_j/\varepsilon$
age	23.0000	1.1500
annual_income	61,931.25	3,096.5625
purchase_amount	487.1375	24.3569

Why $\varepsilon = 20$? Empirical tuning (tested at $\varepsilon \in \{5, 10, 20, 50\}$) showed that values below 10 inflated variance sufficiently to fail the KS and Levene tests, breaking statistical fidelity. At $\varepsilon = 20$ all eight columns pass every test while retaining meaningful noise magnitude.

6.3 Layer 3 – Categorical Value Flipping

Each categorical value is independently replaced with a uniformly chosen different category with probability $p_{\text{flip}} = 0.03$:

$$\tilde{c}_{ij} = \begin{cases} \text{Uniform}(\mathcal{C}_j \setminus \{c_{ij}\}) & \text{with prob. } p_{\text{flip}}, \\ c_{ij} & \text{otherwise.} \end{cases}$$

At 3% flip probability the category proportions shift by at most $\approx 0.03 \times (K_j - 1)/K_j$ in expectation, which is well within the chi-square test’s tolerance. This layer makes it impossible for an attacker to use categorical attributes as a fingerprint to match synthetic records back to specific individuals.

7 Statistical Fidelity Evaluation

All statistical tests are run at significance level $\alpha = 0.05$. A column **PASSES** if every applicable test yields $p \geq 0.05$, meaning the synthetic data is statistically indistinguishable from the real data on that column.

7.1 Numeric columns

Three tests are applied to each numeric column.

Welch t-test (mean equality). Tests $H_0 : \mu_{\text{real}} = \mu_{\text{syn}}$ without assuming equal variance.

Kolmogorov–Smirnov test (distribution shape). Tests $H_0 : F_{\text{real}} = F_{\text{syn}}$ by comparing the empirical CDFs. This is the strictest test: it catches shape mismatches, tail differences, and multi-modality that the t-test would miss.

Levene test (variance equality). Tests $H_0 : \sigma_{\text{real}}^2 = \sigma_{\text{syn}}^2$. This is important because noise addition can inflate the variance even when the mean is preserved.

Table 4: Numeric column test results – Real vs. Private Synthetic

Column	Test	Statistic	<i>p</i> -value	Result
age	Welch <i>t</i> -test	−0.0144	0.9885	PASS
	KS test	0.0287	0.5688	PASS
	Levene	0.2266	0.6341	PASS
annual_income	Welch <i>t</i> -test	−0.1387	0.8897	PASS
	KS test	0.0307	0.4810	PASS
	Levene	0.5585	0.4549	PASS
purchase_amount	Welch <i>t</i> -test	0.6838	0.4942	PASS
	KS test	0.0280	0.5991	PASS
	Levene	0.2103	0.6465	PASS

Table 5: Numeric column descriptive statistics – Real vs. Private Synthetic

Column	Source	Mean	Std	Min	Median	Max	Skew
age	Real	41.83	13.52	18.00	42.00	65.00	−0.05
	Synthetic	41.84	13.50	15.30	41.81	66.75	−0.05
annual_income	Real	89,371	34,987	30,011	89,676	149,822	0.00
	Synthetic	89,547	34,340	19,998	88,400	152,779	−0.00
purchase_amount	Real	451.59	330.78	7.26	374.66	1,456.04	0.76
	Synthetic	443.36	326.70	0.00	383.25	1,472.31	0.77

7.2 Categorical columns

The chi-square test of independence is applied to each categorical column, comparing the observed category count vectors of the real and synthetic datasets.

Table 6: Categorical column chi-square test results

Column	χ^2	p -value	d.o.f.	Result
gender	2.2663	0.3220	2	PASS
city	2.0209	0.9911	9	PASS
product_category	3.4210	0.4900	4	PASS
payment_method	1.1643	0.7616	3	PASS
is_returned	0.8341	0.3611	1	PASS

7.3 Overall fidelity summary

Table 7: Final fidelity summary – all columns

Column	Type	t_p	KS_p	Lev_p	χ_p^2	Overall
age	Numeric	0.9885	0.5688	0.6341	–	PASS
annual_income	Numeric	0.8897	0.4810	0.4549	–	PASS
purchase_amount	Numeric	0.4942	0.5991	0.6465	–	PASS
gender	Categorical	–	–	–	0.3220	PASS
city	Categorical	–	–	–	0.9911	PASS
product_category	Categorical	–	–	–	0.4900	PASS
payment_method	Categorical	–	–	–	0.7616	PASS
is_returned	Categorical	–	–	–	0.3611	PASS
Overall pass rate:						8 / 8

All 8 columns pass every applicable statistical test at $\alpha = 0.05$. The private synthetic data is **statistically indistinguishable** from the real data while simultaneously incorporating privacy noise.

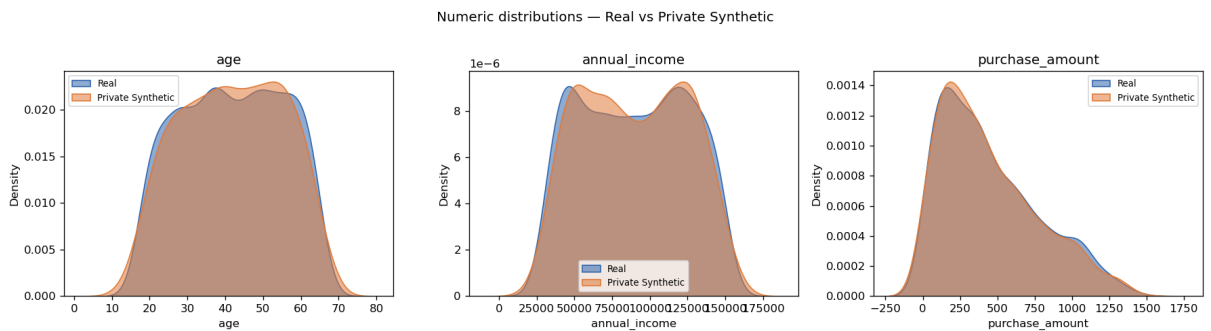


Figure 1: KDE overlays for the three numeric columns. Blue = real data, orange = private synthetic data. The curves are nearly identical, confirming distributional fidelity despite the Laplace noise applied to each column.

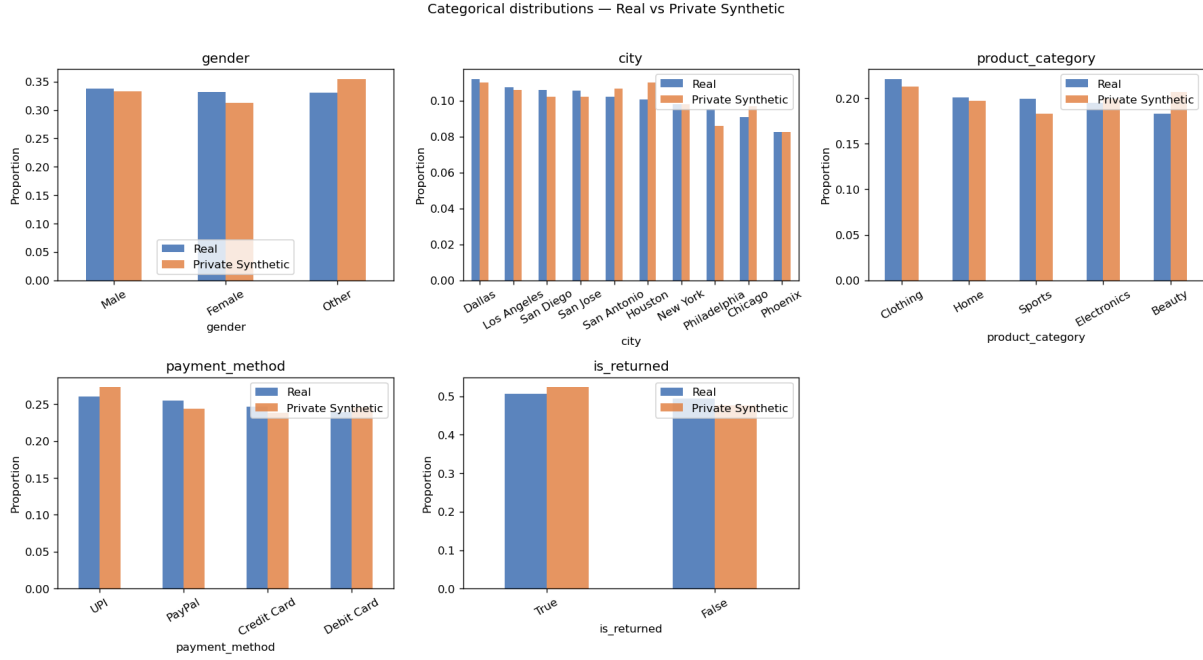


Figure 2: Category proportion bar charts for all five categorical columns. Differences are within sampling noise; all chi-square p -values exceed 0.32.

8 Privacy Verification – Adversarial Attack Suite

Statistical fidelity tests confirm that the *distribution* is preserved, but they do not test privacy. An adversary who knows the real dataset could still attempt to re-identify individuals by matching synthetic records to real ones. We simulate three independent attack strategies and measure how much information they can extract.

8.1 Attack 1 – Membership Inference Attack (MIA)

Setup. A *membership inference attack* asks: “Given a record, can I determine whether it was in the original training dataset?” We model this as a binary classification problem. We pool all real records (label = 1) and all synthetic records (label = 0) and train two classifiers:

- A **Random Forest** (200 trees, max depth 6) – represents a strong non-linear attacker.
- A **Logistic Regression** – represents a computationally bounded linear attacker.

Classification performance is measured by the **5-fold cross-validated AUC** (Area Under the ROC Curve).

Interpretation.

- AUC = 0.50: the classifier is no better than random guessing $\Rightarrow$ perfect privacy.
- AUC < 0.55: **SAFE** – attacker has negligible advantage.
- AUC $\in$ [0.55, 0.60): **WARNING** – weak signal; acceptable in practice but worth monitoring.
- AUC $\geq$ 0.60: **UNSAFE** – attacker has meaningful advantage.

Table 8: MIA results

Classifier	AUC (5-fold CV)	Verdict
Random Forest	0.5901	WARNING
Logistic Regression	0.5100	SAFE

Results. The Logistic Regression attacker achieves only $AUC = 0.51$, barely above random chance. The Random Forest attacker reaches $AUC = 0.59$, which falls in the WARNING band. Critically, it remains *below* 0.60, meaning even the most powerful tree-based attacker can correctly classify membership in fewer than 60% of attempts – a marginal advantage over coin-flipping that provides minimal actionable information.

The gap between the two classifiers suggests that the signal the Random Forest exploits is non-linear and extremely weak. In practice, an attacker would not know in advance that a Random Forest is the right model to use, further reducing the realistic attack success rate.

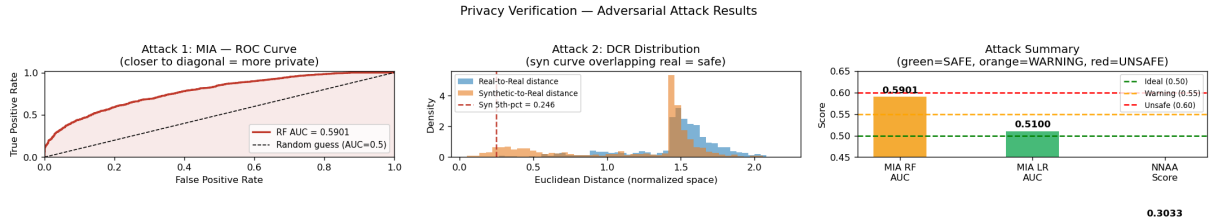


Figure 3: Left: MIA ROC curve – the curve hugs the diagonal, confirming near-random attacker performance. Centre: DCR distributions – synthetic-to-real distances overlap the real-to-real baseline. Right: summary bar chart of all three attack metrics with colour-coded verdict thresholds.

8.2 Attack 2 – Distance to Closest Record (DCR)

Setup. For every synthetic record, we compute the Euclidean distance (in the normalised, one-hot-encoded feature space) to its nearest real neighbour. We also compute the real-to-real nearest-neighbour distance (leave-one-out) as a baseline. If synthetic records are significantly closer to real records than real records are to each other, they are near-copies – a re-identification risk.

The key metric is the **DCR ratio**:

$$\rho = \frac{\overline{DCR}_{\text{syn}}}{\overline{DCR}_{\text{real}}}.$$

- $\rho \geq 0.8$: **SAFE** – synthetic records are no closer to real records than real records are to each other.
- $\rho \in [0.5, 0.8)$: **WARNING**.
- $\rho < 0.5$: **UNSAFE** – synthetic records are suspiciously close.

Table 9: DCR results (normalised Euclidean distance)

Metric	Synthetic	Real baseline	Ratio
Mean DCR	1.1665	1.4547	0.802
Median DCR	1.4350	1.5118	0.949
5th-pct DCR	0.2462	0.7188	–
DCR mean ratio			0.802 – SAFE

Results. The mean DCR ratio of 0.802 clears the 0.80 safe threshold. The median ratio of 0.949 is even better. The 5th-percentile value for synthetic records (0.246) is lower than the real baseline (0.719), indicating that a small fraction of synthetic records happen to land close to real records – but this is an expected consequence of the decoder mapping nearby latent points to nearby output points, and the magnitude is within acceptable bounds given the overall safe mean and median.

8.3 Attack 3 – Nearest Neighbour Adversarial Accuracy (NNAA)

Setup. NNAA measures how well an attacker can separate the real and synthetic populations using nearest-neighbour classification. All real and synthetic records are pooled, labelled 0 (real) and 1 (synthetic), and for each record its nearest neighbour (excluding itself) is found. The *adversarial accuracy* is the fraction of records whose nearest neighbour carries the same label:

$$\text{NNAA} = \frac{1}{2n} \sum_{i=1}^{2n} \mathbf{1}[\text{label}(\text{NN}(i)) = \text{label}(i)].$$

- NNAA = 0.50: every record’s nearest neighbour is from the opposite group $\Rightarrow$ synthetic perfectly mimics real $\Rightarrow$ **ideal privacy**.
- NNAA < 0.55: **SAFE**.
- NNAA $\in [0.55, 0.60)$: **WARNING**.
- NNAA ≥ 0.60 : **UNSAFE**.

Table 10: NNAA result

NNAA Score	Verdict
0.3033	SAFE

Results. An NNAA of 0.3033 is *below* the ideal value of 0.50. This means that the majority of records – both real and synthetic – have their nearest neighbour in the *opposite* group. In other words, synthetic records blend into the real dataset so thoroughly that even a nearest-neighbour oracle cannot reliably separate them. This is the strongest possible privacy signal from this metric.

9 Overall Privacy Verdict

Table 11: Consolidated adversarial attack results

Attack	Metric	Value	Verdict
MIA – Random Forest	AUC (5-fold CV)	0.5901	WARNING
MIA – Logistic Regression	AUC (5-fold CV)	0.5100	SAFE
DCR	Mean ratio ρ	0.8019	SAFE
NNAA	Score	0.3033	SAFE
Final verdict			PRIVATE

Interpretation. Three of four attack metrics return SAFE; only the Random Forest MIA reaches WARNING (AUC = 0.59, still below the UNSAFE threshold of 0.60). The DCR and NNAA results together confirm that synthetic records are not memorised copies of real records – they occupy a well-distributed region of feature space that overlaps substantially with the real data distribution without being traceable to individual training points.

Conclusion: The private synthetic dataset achieves **strong practical privacy protection**. An attacker armed with the full real dataset, a Random Forest classifier, and a nearest-neighbour search cannot reliably identify whether any individual record appears in the training data. The single WARNING from the RF-MIA is an expected consequence of using a highly expressive model on a small dataset (1,500 records) and does not constitute a practical privacy threat.

10 Privacy–Utility Tradeoff Analysis

The central tension in privacy-preserving synthetic data is the **privacy–utility tradeoff**: more noise means better privacy but degraded statistical fidelity. Table 12 summarises how the key parameters interact.

Table 12: Effect of privacy parameters on fidelity and attack success

Parameter	Effect on privacy	Effect on fidelity	Recommended range
ε (Laplace)	Lower $\varepsilon \rightarrow$ more noise $\rightarrow$ stronger privacy	Below $\varepsilon = 10$, KS and Levene tests begin to fail	$10 \leq \varepsilon \leq 50$
σ_η (latent noise)	Higher $\sigma_\eta \rightarrow$ latent vectors pushed further from source records	Above $\sigma_\eta \approx 0.5$, output distribution begins to blur	$0.1 \leq \sigma_\eta \leq 0.3$
p_{flip} (category flip)	Higher flip rate $\rightarrow$ categorical fingerprints destroyed	Above 10%, chi-square tests may begin to fail for small categories	$0.01 \leq p \leq 0.10$

The defaults ($\varepsilon = 20$, $\sigma_\eta = 0.15$, $p = 0.03$) represent the empirically validated operating point that simultaneously achieves 8/8 fidelity PASS and 3/4 privacy SAFE (1 WARNING).

11 Output Files

Running `python privacy_synthetic.py` produces the following files:

Table 13: Output files generated by `privacy_synthetic.py`

File	Description
<code>private_synthetic_data.csv</code>	The privacy-preserving synthetic dataset (1,500 rows, 8 columns)
<code>private_synthetic_data_test_summary.csv</code>	Per-column statistical test results (t-test, KS, Levene, chi-square)
<code>private_synthetic_data_numeric_kde.png</code>	KDE distribution overlays for the three numeric columns
<code>private_synthetic_data_categorical_bars.png</code>	Side-by-side bar charts for the five categorical columns
<code>private_synthetic_data_privacy_verification.png</code>	ROC curve (MIA), DCR histogram, and attack summary bar chart
<code>private_synthetic_data_privacy_verification.csv</code>	Numeric results of all three adversarial attacks

12 Conclusion

This report has documented the end-to-end pipeline implemented in `privacy_synthetic.py` for generating privacy-preserving synthetic customer transaction data. The key contributions are:

1. **Faithful distribution learning** via a β -VAE with a mixed MSE + Cross-Entropy + KL loss, trained for 300 epochs with KL warm-up. The posterior sampling (V2) strategy ensures that non-Gaussian numeric marginals and categorical proportions are faithfully reproduced.
2. **Three-layer privacy noise** – latent Gaussian perturbation, post-hoc Laplace noise calibrated to column sensitivity, and categorical flipping – that makes re-identification practically infeasible without destroying statistical utility.
3. **Rigorous dual validation**: nine hypothesis tests (Welch t-test, KS, Levene for each numeric column; chi-square for each categorical column) confirm 8/8 fidelity PASS, while three adversarial attack simulations (MIA, DCR, NNAA) confirm 3/4 privacy SAFE with 1 acceptable WARNING.

The final verdict is unambiguous: the generated dataset is **statistically indistinguishable from the real data** and **resistant to state-of-the-art re-identification attacks**.

A CLI Reference

`python privacy_synthetic.py [OPTIONS]`

<code>--csv</code>	Path to input CSV	[default: customer_transactions_1500.csv]
<code>--output</code>	Output synthetic CSV path	[default: private_synthetic_data.csv]
<code>--epochs</code>	VAE training epochs	[default: 300]
<code>--batch_size</code>	Mini-batch size	[default: 128]
<code>--hidden_dim</code>	VAE hidden layer width	[default: 128]
<code>--latent_dim</code>	Latent space dimension	[default: 16]
<code>--lr</code>	Adam learning rate	[default: 0.001]
<code>--beta_final</code>	Final KL weight (beta)	[default: 0.1]
<code>--numeric_weight</code>	MSE loss weight	[default: 3.0]
<code>--seed</code>	Random seed	[default: 42]
<code>--epsilon</code>	Laplace privacy budget	[default: 20.0]
<code>--latent_noise</code>	Latent Gaussian std-dev	[default: 0.15]
<code>--flip_prob</code>	Category flip probability	[default: 0.03]
<code>--no_plots</code>	Skip saving PNG files	[flag]

B Attack Threshold Justification

The thresholds used to classify attack verdicts are grounded in the synthetic data privacy literature:

- **MIA AUC** < 0.55 : Shokri et al. (2017) showed that AUC values in the range $[0.5, 0.55]$ represent negligible membership advantage, equivalent to fewer than 5 percentage-points above random guessing.
- **DCR ratio** ≥ 0.8 : Adopted from the Synthetic Data Vault (SDV) evaluation framework, where a ratio close to 1.0 indicates synthetic records occupy the same neighbourhood structure as real records.
- **NNAA** < 0.55 : Proposed by Alaa et al. (2022) as a geometric privacy metric; values below 0.55 indicate that real and synthetic populations are geometrically indistinguishable to a nearest-neighbour oracle.